

1. Database Design

1.1 Orders Table

Attribute	Data Type	Constraint
Order_ID	NUMBER(6)	Primary Key
Customer_ID	NUMBER(5)	Foreign Key
Order_Date	DATE	NOT NULL
Order_Status	VARCHAR2(20)	NOT NULL
Total_Amount	NUMBER(10,2)	NOT NULL

1.2 Order_Items Table

Attribute	Data Type	Constraint
OrderItem_ID	NUMBER(6)	Primary Key
Order_ID	NUMBER(6)	Foreign Key
Product_ID	NUMBER(5)	Foreign Key
Quantity	NUMBER(3)	NOT NULL
Unit_Price	NUMBER(10,2)	NOT NULL

2. Table Relationships

Entity 1	Relationship	Entity 2	Cardinality
Customer	Places	Orders	One-to-Many
Orders	Includes	Order_Items	One-to-Many
Order_Items	Refers to	Product	Many-to-One
Orders	Has	Payment	One-to-One

3. Table Creation

3.1 Create Orders Table

```
SQL> CREATE TABLE Orders (  
    Order_ID NUMBER(6) PRIMARY KEY,  
    Customer_ID NUMBER(5),  
    Order_Date DATE NOT NULL,  
    Order_Status VARCHAR2(20) NOT NULL,  
    Total_Amount NUMBER(10,2) NOT NULL,  
    CONSTRAINT fk_order_customer FOREIGN KEY (Customer_ID)  
        REFERENCES Customer(Customer_ID)  
);  
  
Table created.
```

3.2 Create Order_Items Table

```
SQL> CREATE TABLE Order_Items (  
    OrderItem_ID NUMBER(6) PRIMARY KEY,  
    Order_ID NUMBER(6),  
    Product_ID NUMBER(5),  
    Quantity NUMBER(3) NOT NULL,  
    Unit_Price NUMBER(10,2) NOT NULL,  
    CONSTRAINT fk_item_order FOREIGN KEY (Order_ID)  
        REFERENCES Orders(Order_ID),  
    CONSTRAINT fk_item_product FOREIGN KEY (Product_ID)  
        REFERENCES Product(Product_ID)  
);  
  
Table created.
```

4. Order Data Insertion

```
SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Order_Status, Total_Amount)  
VALUES (501, 101, SYSDATE, 'Delivered', 2499.00);  
1 row created.  
  
SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Order_Status, Total_Amount)  
VALUES (502, 102, SYSDATE, 'Shipped', 1899.00);  
1 row created.  
  
SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Order_Status, Total_Amount)  
VALUES (503, 101, SYSDATE, 'Delivered', 3299.00);  
1 row created.  
  
SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Order_Status, Total_Amount)  
VALUES (504, 103, SYSDATE, 'Pending', 999.00);  
1 row created.
```

5. Order Item Data Insertion

```
SQL> INSERT INTO Order_Items (OrderItem_ID, Order_ID, Product_ID, Quantity, Unit_Price)
VALUES (901, 501, 201, 1, 2499.00);
1 row created.

SQL> INSERT INTO Order_Items (OrderItem_ID, Order_ID, Product_ID, Quantity, Unit_Price)
VALUES (902, 502, 202, 1, 1899.00);
1 row created.

SQL> INSERT INTO Order_Items (OrderItem_ID, Order_ID, Product_ID, Quantity, Unit_Price)
VALUES (903, 503, 203, 2, 1649.50);
1 row created.

SQL> INSERT INTO Order_Items (OrderItem_ID, Order_ID, Product_ID, Quantity, Unit_Price)
VALUES (904, 504, 204, 1, 999.00);
1 row created.
```

6. Retrieve Order Details

```
SQL> SELECT o.Order_ID, c.First_Name || ' ' || c.Last_Name AS Customer_Name,
o.Order_Date, o.Order_Status, o.Total_Amount
FROM Orders o JOIN Customer c ON o.Customer_ID = c.Customer_ID
ORDER BY o.Order_Date DESC;
```

ORDER_ID	CUSTOMER_NAME	ORDER_DATE	ORDER_STATUS	TOTAL_AMOUNT
504	Arjun Mehta	26-SEP-26	Pending	999.00
503	Priya Sharma	26-SEP-26	Delivered	3299.00
502	Karthik Raj	26-SEP-26	Shipped	1899.00
501	Priya Sharma	26-SEP-26	Delivered	2499.00

7. Calculate Total Order Value

This uses the SUM() aggregate function.

```
SQL> SELECT Customer_ID, SUM(Total_Amount) AS Total_Spent
FROM Orders
GROUP BY Customer_ID;
```

CUSTOMER_ID	TOTAL_SPENT
101	5798.00
102	1899.00
103	999.00

8. Identify High-Value Orders

Customers whose total order value is 2000 or above can be displayed using HAVING.

```
SQL> SELECT Customer_ID, SUM(Total_Amount) AS Total_Spent
      FROM Orders
      GROUP BY Customer_ID
      HAVING SUM(Total_Amount) >= 2000
      ORDER BY Total_Spent DESC;
```

CUSTOMER_ID	TOTAL_SPENT
-----	-----
101	5798.00

9. Order Summary Report

A combined report can be generated using the Orders and Order_Items tables.

```
SQL> SELECT o.Customer_ID, COUNT(o.Order_ID) AS Total_Orders,
      SUM(o.Total_Amount) AS Total_Value, MAX(o.Total_Amount) AS Highest_Order,
      MIN(o.Total_Amount) AS Lowest_Order
      FROM Orders o
      GROUP BY o.Customer_ID
      ORDER BY Total_Value DESC;
```

CUSTOMER_ID	TOTAL_ORDERS	TOTAL_VALUE	HIGHEST_ORDER	LOWEST_ORDER
-----	-----	-----	-----	-----
101	2	5798.00	3299.00	2499.00
102	1	1899.00	1899.00	1899.00
103	1	999.00	999.00	999.00